

Informe técnico — Sistema inteligente para reparar inconsistencias en registros

Los números provienen de la instancia principal (semilla 42) salvo la §7.3, que mide la robustez del LLM sobre 3 semillas (42, 123, 789). Todo es reproducible con `run_pipeline.py` y los scripts de `experiments/` (ver §6 y el Apéndice). Detección estructural y optimización son deterministas; la detección con LLM depende del modelo (gemma4 vía Ollama).

1. Descripción del problema

Dado un conjunto de hechos sobre registros académicos —unos estructurados (estudiantes, profesores, exámenes, resultados, recalificaciones) y otros textuales (comentarios de los profesores)— que pueden contener contradicciones, el objetivo es encontrar un conjunto mínimo de modificaciones que restaure la coherencia global del historial.

El problema tiene dos retos acoplados:

1. Detección. Identificar las inconsistencias. Algunas son verificables con reglas duras (una nota fuera de `[0,20]` , un examen con fecha futura, un resultado que referencia a un estudiante inexistente). Otras son semánticas y solo se aprecian entendiendo el lenguaje: un comentario elogioso junto a una nota de suspenso, o un comentario que describe una asignatura distinta a la del examen. Estas últimas requieren un LLM.
2. Reparación. Elegir *qué* cambiar. Como cada inconsistencia admite varias reparaciones posibles (con distinto coste) y una misma reparación puede resolver varias inconsistencias a la vez, hay que optimizar: minimizar el coste total de las modificaciones sujeto a que todas las inconsistencias queden cubiertas y a que no se introduzcan nuevas (coherencia global).

El sistema combina, por tanto, detección híbrida (reglas + LLM) con optimización combinatoria (CP-SAT y metaheurísticas).

2. Modelado formal

2.1 Hechos y restricciones

Sea un dataset $D = (E, P, X, R, G, T)$ con conjuntos de estudiantes E , profesores P , exámenes X , resultados R , recalificaciones G y reportes textuales T .

Sobre D se define un conjunto de restricciones de integridad $C = \{c_1, \dots, c_k\}$. Cada c_j es un predicado sobre una o más entidades; por ejemplo:

- $c_{\text{nota}}(r): 0 \leq \text{nota}(r) \leq 20$
- $c_{\text{edad}}(e): |\text{edad}(e) - (\text{año_actual} - \text{año_nacimiento}(e))| \leq 1$
- $c_{\text{curso}}(e): \text{edad}(e) \in \text{rango_edad}(\text{curso}(e))$
- $c_{\text{ghost}}(r): \text{estudiante}(r) \in E \wedge \text{examen}(r) \in X$
- $c_{\text{sem}}(t)$: el comentario t es semánticamente coherente con $(\text{nota}, \text{asistencia}, \text{edad}, \text{asignatura})$

Una inconsistencia es una instancia de restricción violada. Llamamos $\text{Inconsistencias}(D)$ al conjunto de todas las inconsistencias presentes en D (qué restricción se viola y sobre qué entidades).

2.2 Reparaciones

Una reparación p es una modificación del dataset (cambiar un campo, eliminar un registro, etc.). Cada reparación tiene un costo $\text{costo}(p)$ —un entero positivo que modela el esfuerzo o riesgo de esa edición (p.ej. corregir una nota cuesta 1; eliminar un registro cuesta 5). Llamamos REP al conjunto de todas las reparaciones candidatas.

Para cada inconsistencia i , definimos $\text{Cubre}(i)$ = el conjunto de reparaciones candidatas que resuelven esa inconsistencia i .

2.3 Problema de optimización

Buscamos el subconjunto de reparaciones S (con $S \subseteq \text{REP}$) que resuelve esto:

```
minimizar      la suma de los costos de las reparaciones elegidas:
                
$$\sum_{p \in S} \text{costo}(p)$$


sujeto a:
(1) cobertura      cada inconsistencia debe quedar resuelta por al menos
                    una reparación elegida.
(2) coherencia global  tras aplicar todas las reparaciones elegidas, el dataset
                        no debe contener ninguna inconsistencia.
```

Escrito de forma compacta (y la lectura de cada símbolo):

$$\begin{array}{ll} \min & \sum_{p \in S} \text{costo}(p) \\ \text{s.a.} & S \cap \text{Cubre}(i) \neq \emptyset \quad \text{para toda inconsistencia } i \in \text{Inconsistencias}(D) \\ & \text{Inconsistencias}(\text{aplicar}(D, S)) = \emptyset \end{array}$$

donde:

- $\sum_{p \in S} \text{costo}(p)$ = "la suma de $\text{costo}(p)$ para cada reparación p que está en S ".
- $S \cap \text{Cubre}(i) \neq \emptyset$ = " S y $\text{Cubre}(i)$ tienen al menos una reparación en común" → es decir, elegí al menos una reparación que resuelve la inconsistencia i .
- para toda $i \in \text{Inconsistencias}(D)$ = "esto debe cumplirse para cada inconsistencia del dataset".
- $\text{aplicar}(D, S)$ = "el dataset que queda después de aplicar a D todas las reparaciones de S ".
- $\text{Inconsistencias}(\dots) = \emptyset$ = "ese dataset no tiene ninguna inconsistencia" ($\emptyset$ = conjunto vacío).

La restricción (1) cobertura es exactamente un problema clásico llamado cubrimiento de conjuntos ponderado de costo mínimo (*weighted minimum set cover / hitting set*), que es NP-difícil. La restricción (2) coherencia global exige además que el dataset reparado quede totalmente limpio, sin inconsistencias nuevas introducidas como efecto colateral de una reparación (una *cascada*).

2.4 Formulación CP-SAT

Para resolverlo con el solver, se define una variable binaria $x_p \in \{0,1\}$ por cada reparación candidata: $x_p = 1$ significa "la reparación p se aplica", y $x_p = 0$ significa "no se aplica". El modelo queda:

$$\begin{array}{ll} \min & \sum_p \text{costo}(p) \cdot x_p & (\text{minimizar el costo total}) \\ \text{s.a.} & \sum_{p \in \text{Cubre}(i)} x_p \geq 1 & \text{para toda } i \quad (\text{cada inconsistencia, cubierta por } \geq 1 \text{ reparación aplicada}) \\ & x_p \in \{0,1\} \end{array}$$

La coherencia global no se codifica como restricción del modelo (ver §4.4): se garantiza por construcción (reparaciones auto-consistentes) y se verifica a posteriori re-detectando sobre el dataset reparado $\text{aplicar}(D, S)$.

3. Dataset

3.1 Naturaleza

Dataset sintético generado por `data/seed.py` (semilla fija para reproducibilidad). Escala de la instancia principal (semilla 42):

Entidad	Cantidad
Estudiantes	50
Profesores	6
Exámenes	40
Resultados	159
Recalificaciones	48
Reportes de profesor	156

3.2 Proceso de generación

La generación sigue el patrón "base limpia + inyección controlada":

1. Base coherente. Primero se generan entidades que satisfacen *todas* las restricciones (edades acordes al curso, créditos en rango, notas en `[0,20]`, pares (estudiante, examen) únicos, fechas pasadas, etc.).
2. Inyección determinista. Se inyecta un número exacto y conocido de cada tipo de inconsistencia, sobre conjuntos de entidades disjuntos y de forma no cascada, de modo que `inyectadas == reales`. Dos decisiones de diseño clave garantizan esto: - La inconsistencia *edad*↔*nacimiento* se inyecta desplazando la fecha de nacimiento (no la edad), para que la edad siga siendo válida para el curso y no se dispare además una inconsistencia *curso*↔*edad*. - La inconsistencia *curso*↔*edad* re-ajusta los créditos al rango del nuevo curso, para no disparar además una inconsistencia de créditos.
3. Ground truth por ID. El generador registra en `metadata` qué entidad exacta lleva cada inconsistencia (`structural_ground_truth`) y, para las textuales, el `report_id`, el tipo y la clase de redacción (`textual_ground_truth`). Esto permite evaluación a nivel de instancia (no solo por conteo).

3.3 Inconsistencias inyectadas (semilla 42)

Tipo	Cantidad	Detector
age_birth_mismatch	5	estructural
course_age_mismatch	4	estructural
credits_mismatch	4	estructural
invalid_grade	6	estructural
ghost_student	3	estructural
ghost_exam	3	estructural
professor_subject_mismatch	4	estructural
future_exam_date	3	estructural
duplicate_result	3	estructural
orphan_regrade	3	estructural
textual_inconsistency	19	textual (LLM)
Total	57	

Las 19 textuales se subdividen por clase de redacción, que es central para medir el valor del LLM:

- Canónica (4): usan frases del catálogo que las reglas por palabras clave sí pueden cazar.
- Parafraseada (11): misma contradicción semántica (nota↔comentario, asistencia↔comentario, edad↔comentario) pero con vocabulario ausente de los markers — solo detectable razonando sobre el significado.
- Global (4): el comentario describe una asignatura distinta a la del examen (p.ej. "ecuaciones y cálculo" en un examen de física). Es una contradicción entre dos elementos del historial (reporte ↔ examen), invisible tanto para las reglas como para un chequeo local que no mire la asignatura.

3.4 ¿Por qué cumple las condiciones del problema?

- Contiene hechos estructurados y textuales con contradicciones reales de ambos tipos (verificables por reglas y semánticas).
 - Tiene ground truth exacto (por ID y por tipo), lo que permite medir detección y reparación con rigor.
 - Las inconsistencias textuales están diseñadas para discriminar reglas vs LLM: la presencia de casos parafraseados y globales hace que un detector puramente léxico fracase y que solo un evaluador semántico (LLM) los resuelva — exactamente el escenario que la consigna exige evaluar.
 - Las reparaciones se solapan (una reparación raíz cubre varias inconsistencias), lo que da sustancia al problema de optimización.
-

4. Diseño del algoritmo

El sistema es un pipeline de 5 etapas (`run_pipeline.py`):

```
seed.py → structural_rules.py → llm_detector.py → repair_optimizer.py → validate.py / ev
```

4.1 Detección estructural (`detector/structural_rules.py`)

Son diez detectores deterministas, uno por cada restricción dura. Cada detector es una función que recorre las entidades relevantes y evalúa un predicado booleano; si el predicado se viola, emite una inconsistencia con los IDs de las entidades implicadas, el campo afectado y una reparación sugerida. Por ejemplo:

- *nota inválida*: recorre los resultados y comprueba $0 \leq \text{nota} \leq 20$.
- *edad↔nacimiento*: para cada estudiante compara la edad almacenada con la que se deduce del año de nacimiento (`año_actual - año_nacimiento`) y marca si difieren en más de 1.
- *estudiante/examen fantasma*: comprueba que el `estudiante_id` / `examen_id` de cada resultado exista en el conjunto de estudiantes / exámenes.
- *resultado duplicado*: detecta pares (estudiante, examen) repetidos usando un conjunto de pares ya vistos.
- *profesor↔asignatura*: comprueba que la asignatura del examen esté entre las que imparte su profesor.

Es exacto, lineal en el número de entidades y totalmente explicable (no hay aprendizaje ni azar): la salida es siempre la misma y se puede justificar regla a regla. Estas reglas son la *verdad de referencia* para lo verificable objetivamente; lo que no se puede verificar con una regla (la coherencia semántica del lenguaje) queda para el LLM.

4.2 Detección textual (`detector/llm_detector.py`)

Hay tres modos seleccionables con la variable `DETECTOR_MODE` :

- `rule` (baseline por palabras clave). Para cada categoría de inconsistencia hay una lista de frases-marcador; el detector comprueba si el comentario contiene alguna de esas subcadenas y, combinándolo con condiciones numéricas sobre la nota o la asistencia, decide. Es rápido y preciso con la redacción literal, pero ciego a cualquier paráfrasis (si la frase no está en la lista, no la ve) y a lo global (no mira la asignatura).
- `llm` (el LLM es el juez semántico único). Para cada reporte se construye un *prompt* con los hechos objetivos (edad, asistencia, curso, asignatura del examen y nota) más el comentario y la definición de cada categoría. Se le da el *significado* de las escalas (qué es una nota alta, qué es asistencia baja) pero no los umbrales-regla, para que razone la coherencia. El prompt se envía al modelo (gemma4) servido por Ollama vía HTTP (`format=json` , `temperature=0` para que sea reproducible). El modelo devuelve un JSON con su veredicto

(`es_coherente` , `tipo_inconsistencia` , `comentario_reparado`); ese veredicto es la detección y no se sobrescribe con reglas.

- `hybrid` (reglas + LLM). Las reglas cazan lo obvio; los reportes que ellas consideran coherentes se escalan al LLM, que añade recall sobre lo que las reglas no ven.

La salida cruda del LLM se valida con Pydantic antes de usarse (ver §5): si el modelo devuelve un JSON malformado o una etiqueta fuera del catálogo, el sistema degrada de forma segura en lugar de romperse.

4.3 Optimización de reparaciones (`detector/repair_optimizer.py`)

Las tres variantes resuelven el mismo problema (el cubrimiento ponderado de la §2.4), pero con estrategias muy distintas:

CP-SAT (exacto, OR-Tools). CP-SAT es un solver de programación por restricciones apoyado en un motor SAT. Se le da el modelo de la §2.4: una variable binaria por cada reparación (aplicar / no aplicar), una restricción por cada inconsistencia ("al menos una de sus reparaciones debe aplicarse") y la función objetivo (minimizar el coste total). El solver hace una búsqueda inteligente (ramificación y acotamiento + propagación de restricciones + aprendizaje de cláusulas) que poda enormes partes del espacio de soluciones sin tener que enumerarlas, y demuestra la optimalidad: sabe cuándo ninguna solución mejor es posible. Garantiza el óptimo, a costa de un tiempo que en el peor caso es exponencial.

Greedy (heurístico voraz). Construye la solución paso a paso: en cada iteración elige la reparación con el mejor ratio `inconsistencias nuevas que cubre ÷ coste` , la añade, marca esas inconsistencias como cubiertas y repite hasta cubrir todas. Es muy rápido (tiempo polinómico), pero miope: cada decisión parece la mejor *en ese momento*, lo que puede llevar a un total globalmente malo (es exactamente lo que explota la "trampa-greedy" de §7.6).

Recocido simulado (SA, metaheurístico). Inspirado en el enfriamiento de un metal: explora el espacio de soluciones aceptando a veces movimientos peores para escapar de óptimos locales. Arranca desde una solución inicial (*warm-start* = la de greedy) y repite: propone un vecino (un cambio pequeño: añadir o quitar una reparación, o añadir una candidata para una inconsistencia al azar) y lo evalúa. Si el vecino es mejor, lo acepta; si es peor, lo acepta con probabilidad $e^{(-\Delta/T)}$ —con temperatura `T` alta acepta empeoramientos con facilidad (explora), con `T` baja se vuelve exigente (explota)—. La temperatura baja gradualmente (enfriamiento), de modo que al principio explora mucho y al final afina. La función objetivo que minimiza combina el coste con una penalización fuerte por inconsistencias sin cubrir.

Más allá de qué algoritmo se use, la estructura del problema incluye un caso especial que los tres aprovechan: las reparaciones raíz; es una forma de modelar el solapamiento entre inconsistencias.

Una reparación "raíz" (p.ej. corregir la asignatura de un examen mal asignado) puede cubrir a la vez la inconsistencia estructural del examen, las recalificaciones derivadas y los comentarios textuales ligados a ese examen. CP-SAT (y también greedy y SA) lo aprovecha de forma natural:

la reparación raíz se ofrece como alternativa a *todas* las inconsistencias que cubre, y por tanto el optimizador puede elegir una sola reparación cara que resuelve muchas en vez de varias baratas. Este es el punto donde el dataset real tiene solapamiento genuino entre reparaciones.

4.4 Coherencia global: reparaciones auto-consistentes + verificación

En lugar de codificar las cascadas como restricciones (ver §8), se garantiza la coherencia global con dos mecanismos:

1. Reparaciones auto-consistentes. Cada reparación se diseña para no romper otros invariantes: corregir *edad*↔*nacimiento* edita la fecha de nacimiento (no la edad, que participa en otras restricciones); cambiar el curso ajusta los créditos al rango del nuevo curso en el mismo paso.
2. Verificación post-reparación. Tras aplicar `S`, el optimizador re-detecta estructuralmente sobre el dataset reparado y registra en `repair_plan.json`: `json "verification": {"global_coherence_verified": true, "residual_structural_issues": 0, "residual_by_type": {}}` Así "coherencia global restaurada" es una propiedad probada en cada corrida, no una suposición. (`validate.py` repite esta comprobación de forma independiente.)

5. Rol del LLM en el sistema

El LLM (servido localmente con Ollama; modelo gemma4 8B) es el evaluador de consistencia semántica, la pieza que la consigna exige y que ningún método léxico puede sustituir.

- Entrada. Para cada reporte se le pasan los hechos objetivos (edad, asistencia, curso, asignatura del examen, nota) y el comentario. Se le da el *significado* de las escalas (qué es una nota alta, qué es asistencia baja) pero no los umbrales-regla, para que razone la coherencia en vez de ejecutar condiciones predefinidas.
 - Tarea. Decidir si el comentario es coherente y, si no, clasificar el tipo y reescribirlo de forma coherente. Detecta dos niveles de consistencia semántica:
 - Local: comentario ↔ nota/asistencia/edad del propio estudiante.
 - Global: comentario ↔ asignatura del examen (consistencia *entre elementos del historial*).
 - Validación con Pydantic. Tres modelos tipados blindan la frontera con el LLM: `LLMOutputRaw` (parseo laxo del JSON crudo), `LLMOutputValidated` (esquema estricto: el tipo debe pertenecer a un `Literal[...]` cerrado, el comentario reparado no vacío) y `TextualRecord` (registro final). Importante: Pydantic valida la forma de la respuesta (que sea un JSON con tipos y etiquetas válidas), no la corrección del veredicto. La corrección semántica la aporta el LLM; su calidad se mide a posteriori contra el ground truth (§7). Ante un JSON malformado, el sistema degrada de forma segura a un *fallback* coherente.
-

6. Metodología experimental

6.0 Métricas de evaluación (qué son TP, FP, FN)

Cada detección se compara contra el ground truth y se clasifica en una de cuatro categorías (de la matriz de confusión):

	Es realmente inconsistente	Es realmente coherente
El sistema lo marcó inconsistente	TP (verdadero positivo) ✓ acierto	FP (falso positivo) X falsa alarma
El sistema lo marcó coherente	FN (falso negativo) X se le escapó	TN (verdadero negativo) ✓

- TP — inconsistencia real que el sistema sí detectó.
- FP — algo coherente que el sistema marcó como inconsistente (falsa alarma).
- FN — inconsistencia real que el sistema no detectó (se le pasó).

A partir de ellas se calculan las métricas:

- Precisión $= TP / (TP + FP)$ — de todo lo que se marcó, qué fracción era real. Penaliza las falsas alarmas.
- Recall (exhaustividad) $= TP / (TP + FN)$ — de todo lo que era real, qué fracción se encontró. Penaliza lo que se escapa.
- F1 $= 2 \cdot P \cdot R / (P + R)$ — media armónica de precisión y recall; resume ambas en un solo número (alto solo si las dos son altas).
- MCC (coeficiente de correlación de Matthews) $\in [-1, 1]$ — 1 = perfecto, 0 = azar, negativo = peor que azar. Es robusto ante el desbalance (hay muchos registros coherentes y pocas inconsistencias), donde la precisión/recall pueden engañar.

Evaluación por ID (no por conteo). Una detección cuenta como TP solo si cae sobre la entidad exacta que es inconsistente (el `student_id`, `result_id`, `exam_id` o `report_id` correcto). No basta con acertar el *número* de inconsistencias: marcar 5 estudiantes equivocados no es un acierto. Esto evita inflar los resultados y es más estricto que una evaluación por conteo.

6.1 Experimentos

La instancia principal usa la semilla 42; la robustez se mide sobre 3 semillas.

1. Detección estructural. `evaluate.py` compara las detecciones contra `structural_ground_truth` por ID (no por conteo): $TP = |GT \cap detectados|$, $FP = detectados - GT$, $FN = GT - detectados$. Métricas: precisión, recall, F1 y MCC por tipo y global.
2. Detección textual — reglas vs LLM. Se ejecuta el detector en modo `rule` y en modo `llm` (gemma4) y se evalúa por `report_id` contra `textual_ground_truth`, desglosando el

recall por clase de redacción (canónica / parafraseada / global) —

`experiments/textual_eval.py`. Por el coste de inferencia local (~3 tokens/seg en CPU ⇒ ~50–75 s/llamada con gemma4), el LLM se corre sobre un subconjunto de evaluación = los 19 reportes con ground truth + 15 coherentes muestreados (flag `--eval-subset`). El recall queda exacto (incluye todo el GT) y la precisión queda estimada sobre la muestra.

3. Robustez del LLM entre instancias (`experiments/llm_robustness.py`). Se repite la detección con LLM sobre 3 semillas independientes (42, 123, 789), cada una en su subconjunto, y se agrega el recall por clase de redacción (media y rango) para comprobar que el LLM generaliza y no acierta por azar.
4. Comparación de configuraciones (`experiments/compare_detectors.py`). Tabla `rule` / `hybrid` / `llm` sobre el mismo subconjunto; `hybrid` se deriva como la unión `rule` $\cup$ `llm`.
5. Comparación de optimizadores (`experiments/hard_instances.py`). Como el dataset real es "fácil" (las 3 variantes empatan), se generan familias de instancias difíciles de hitting set ponderado, en el mismo formato que consumen los solvers reales: una familia aleatoria con solapamiento y una trampa-greedy (caso clásico donde greedy $\approx \ln(n) \cdot \text{óptimo}$). Se reporta coste, *gap* respecto al óptimo de CP-SAT y tiempo.
6. Ajuste de la metaheurística (`experiments/sa_tuning.py`). Barrido de temperatura inicial y operador de vecindad de SA sobre la trampa-greedy, para estudiar si y cómo escapa del óptimo local.
7. Verificación de coherencia global. En cada corrida del optimizador se re-detecta sobre el dataset reparado (§4.4).

7. Resultados y análisis

7.1 Detección estructural

Perfecta en los 10 tipos: precisión = recall = F1 = 1.0, MCC = 1.0 (TP=38, FP=0, FN=0). El diseño "base limpia + inyección disjunta no cascada" elimina los falsos positivos accidentales que un dataset aleatorio produciría (p.ej. duplicados o desajustes de curso fortuitos).

7.2 Detección textual — reglas vs LLM

Sobre el subconjunto de evaluación (34 reportes, 19 con inconsistencia):

Detector	TP	FP	FN	Precisión	Recall	F1
rule	4	0	15	1.000	0.211	0.348
llm (gemma4)	17	4	2	0.810	0.895	0.850

Recall por clase de redacción (el resultado central del proyecto):

Clase	reglas	LLM (gemma4)
canónica (léxica)	4/4 (100%)	4/4 (100%)
parafraseada (semántica local)	0/11 (0%)	10/11 (91%)
global (comentario↔asignatura)	0/4 (0%)	3/4 (75%)

Las reglas tienen precisión perfecta pero recall ínfimo: solo ven coincidencias léxicas literales y son completamente ciegas a las paráfrasis (0/11) y a las contradicciones globales (0/4). El LLM hace razonamiento semántico real en dos niveles y eleva el F1 de 0.35 a 0.85. Esto demuestra, con métrica, *por qué* la consigna exige un LLM: es lo único que generaliza más allá del catálogo de palabras. El único caso global no detectado (comentario de *química* en examen de *biología*) es un *borderline* razonable por el solapamiento temático de ambas ciencias.

7.3 Robustez del LLM entre instancias (multi-semilla)

Para comprobar que el desempeño del LLM no es un artefacto de la semilla 42, se repitió la detección textual con gemma4 sobre 3 datasets independientes (semillas 42, 123, 789), cada uno evaluado sobre su propio subconjunto (`experiments/llm_robustness.py`):

seed	Prec	Recall	F1	canónica	parafr.	global
42	1.000	0.895	0.944	100%	91%	75%
123	0.857	0.947	0.900	83%	100%	100%
789	0.857	0.947	0.900	100%	89%	100%
media [min–max]	0.905 [0.86–1.0]	0.930 [0.90–0.95]	0.915 [0.90–0.94]	94%	93%	92%

El desempeño es estable (F1 entre 0.90 y 0.94): el LLM generaliza a instancias distintas, no acierta por suerte en una. El recall es la métrica robusta (0.93, rango estrecho [0.90–0.95]) porque su denominador —los reportes del ground truth— es fijo. La precisión varía más (0.86–1.0) y aquí está estimada sobre una muestra pequeña de coherentes (subconjunto reducido), por lo que el 1.0 de la semilla 42 es optimista (con más coherentes ronda 0.81, §7.2). El razonamiento semántico se sostiene en ambos niveles a través de las semillas: parafraseada 89–100%, global 75–100%.

7.4 Comparación de configuraciones

Modo	Prec	Recall	F1
rule	1.000	0.211	0.348
hybrid	0.810	0.895	0.850
llm	0.810	0.895	0.850

hybrid == llm porque el LLM cazó todo lo que cazaron las reglas y más; la unión no añade nada. Conclusión: con un LLM suficientemente capaz, el modo híbrido no aporta sobre el LLM solo (solo serviría si las reglas cubrieran algún caso que el LLM falla).

7.5 Optimización: reparación del dataset real

CP-SAT cubre las 48 inconsistencias con 46 reparaciones, coste total 102, y la coherencia global queda verificada (0 inconsistencias residuales). Sobre el dataset real, greedy y SA producen el mismo resultado que CP-SAT (la instancia es fácil), por lo que la comparación de optimizadores se hace sobre instancias difíciles.

7.6 Comparación de optimizadores (instancias difíciles)

Familia aleatoria (gap medio respecto al óptimo de CP-SAT): greedy +10.6 %, SA +7.9 %. CP-SAT demuestra optimalidad en todas las instancias.

Familia trampa-greedy (un repair óptimo cubre todo; greedy cae en la escalera):

n	CP-SAT (óptimo)	greedy	SA (def.)
16	15000	+125%	+125%
32	15000	+171%	+171%
64	15000	+216%	+216%
128	15000	+262%	+262%

El *gap* de greedy crece con n siguiendo $\sim \ln(n)$, exactamente como predice la teoría de aproximación de set-cover. SA por defecto hereda la mala solución de greedy (su warm-start) y no escapa.

7.7 Ajuste de la metaheurística (trampa $n=64$, óptimo=15000)

Configuración SA	coste medio	gap	escapó
default ($T_0=25$)	47437	+216%	0/5
caliente ($T_0=2000$)	47437	+216%	0/5
muy caliente ($T_0=20000$)	16407	+9%	0/5
+ poda de redundancia ($T_0=25$)	15000	+0%	5/5
+ poda de redundancia ($T_0=2000$)	15000	+0%	5/5

Análisis. Subir la temperatura ayuda pero es frágil (mejora a +9% pero no alcanza el óptimo). En cambio, añadir un operador de poda de redundancia (eliminar reparaciones que dejan de ser necesarias) convierte "añadir la reparación raíz" en un movimiento *cuesta abajo* y SA escapa de la trampa de forma robusta (5/5). Lección de comportamiento: la calidad de una metaheurística

depende tanto del operador de vecindad como de la temperatura; CP-SAT, al ser exacto, encuentra el óptimo siempre sin necesidad de ajuste.

7.8 Comportamiento del LLM local

Hallazgo relevante de ingeniería: la inferencia local es generation-bound (~3 tokens/seg en CPU), de ahí ~50–75 s por llamada con gemma4. Además, el modelo más pequeño probado (neural-chat 7B) sobre-detecta (marca como inconsistentes reportes coherentes), mientras que gemma4 8B (con razonamiento interno) acierta de forma fiable. Conclusión: la calidad de la evaluación semántica depende fuertemente del modelo, y hay un compromiso explícito precisión ↔ latencia.

8. Limitaciones y posibles mejoras

Limitaciones

1. Coste de inferencia del LLM. ~50–75 s/llamada en CPU obliga a evaluar sobre un subconjunto; un pase completo de los 156 reportes tarda horas.
2. Coherencia global fuera del modelo. Las cascadas se evitan con reparaciones auto-consistentes y se *verifican* a posteriori, pero no se modelan como restricciones dentro de CP-SAT; la minimalidad es, por tanto, minimalidad sobre las inconsistencias *detectadas*, no un óptimo global demostrado frente a cualquier cascada posible.
3. Consistencia global limitada a comentario↔signatura. No se cubren aún contradicciones entre *múltiples comentarios* del mismo estudiante o entre comentario y recalificaciones.
4. Costes de reparación heurísticos. Los pesos (1, 2, 3, 5...) son razonables pero no están calibrados empíricamente.
5. Dataset sintético. Reproducible y con ground truth, pero no captura todo el ruido de datos reales.

Posibles mejoras

1. Modelar las cascadas en CP-SAT (restricciones de implicación entre reparaciones) para un óptimo global demostrable.
 2. Consistencia global más rica: pasar al LLM el *historial completo* del estudiante (varios comentarios, recalificaciones, profesor) y detectar contradicciones entre registros.
 3. Acelerar el LLM: cuantización/GPU, procesamiento por lotes, o un modelo destilado; e introducir *few-shot* para subir la precisión de modelos pequeños.
 4. Calibrar los costes con criterio de dominio o aprendiéndolos.
 5. Validar sobre datos reales (anonimizar registros académicos reales) y medir transferencia.
 6. Verificación textual post-reparación con el LLM (hoy solo se re-verifica la parte estructural).
-

Apéndice — Cómo reproducir

```
# Pipeline estructural completo + evaluación (sin LLM, rápido)
python run_pipeline.py --skip-textual          # usa modo rule por defecto

# Detección textual con LLM (gemma4) sobre subconjunto de evaluación
DETECTOR_MODE=llm OLLAMA_MODEL=gemma4 python detector/llm_detector.py \
    --output data/textual_llm.json --eval-subset 15 --timeout 180

# Evaluación textual por ID y por redacción
python experiments/textual_eval.py --textual data/textual_llm.json --label llm
python experiments/compare_detectors.py      # tabla rule/hybrid/llm

# Robustez del LLM entre instancias (3 semillas, ~50-60 min en CPU)
python experiments/llm_robustness.py --seeds 42 123 789 --subset 6

# Comparación de optimizadores y ajuste de SA (rápidos, sin LLM)
python experiments/hard_instances.py
python experiments/sa_tuning.py
```

Artefactos generados: `data/evaluation_report.json`, `data/repair_plan.json` (incluye el bloque `verification`), `data/config_comparison_light.json`, `data/llm_robustness_report.json`, `data/hard_instances_report.json`, `data/sa_tuning_report.json`.